README: A Primer for the MSARCH Project

Proteins, Drugs, Network Proximity, QUBO, QAOA, and cuQuantum
for readers who know SIMD, assembly, and CUDA

September 2026

How to use this document

The main specification (`main.tex`) explains the project in plain terms. This primer goes one level deeper on the background you may not have: the biology behind the graph, the statistics behind the z-score, how we judge whether the results mean anything, and the quantum computing behind the optional part. Every concept is tied back to arrays, bits, and kernels. Section 6 maps each pipeline module to these concepts, and Section 7 is a glossary.

Contents

1 Biology you need (and only that) 2
- 1.1 Genes, proteins, and why proteins are the nodes . . . 2
- 1.2 Protein–protein interactions and the interactome . . . 2
- 1.3 Alzheimer's disease and the disease module . . . 2
- 1.4 Drugs and targets . . . 2
- 1.5 Drug repurposing . . . 3

2 Network proximity, step by step 3
- 2.1 Shortest paths . . . 3
- 2.2 Bit-parallel multi-source BFS . . . 3
- 2.3 Proximity . . . 3
- 2.4 Why random samples, and what a z-score is . . . 4
- 2.5 Separation between two drugs . . . 4

3 How we know if the results mean anything 4
- 3.1 Positives and baselines . . . 4
- 3.2 AUROC, AUPRC, and enrichment . . . 4

4 Optimization: QUBO 5

5 Quantum computing, just enough 5
- 5.1 Qubits and the state vector . . . 5
- 5.2 Gates . . . 5
- 5.3 QAOA . . . 6
- 5.4 cuQuantum and CUDA-Q . . . 6

6 Where each concept lives in the pipeline **6**

7 Glossary **7**

1 Biology you need (and only that)

1.1 Genes, proteins, and why proteins are the nodes

DNA is a long string over $\{A, C, G, T\}$. A **gene** is a region of it that encodes a **protein**. Humans have about 20,000 protein-coding genes. Proteins do most of the work in a cell: they are enzymes, signals, receptors, and structural parts. We use gene names and protein names almost interchangeably, because each gene maps to (mostly) one protein; converting between naming systems is a real chore in data preparation, not a concept.

1.2 Protein–protein interactions and the interactome

Proteins act by physically binding each other. Laboratories have measured hundreds of thousands of such pairs, and databases like BioGRID and STRING collect them. Treat every protein as a node and every measured interaction as an undirected edge: that graph is the **interactome**.

Hardware analogy

It is a sparse, undirected, unweighted graph with about 18,000 nodes and a few hundred thousand edges, stored in CSR form (`rowptr`, `col`). Its degree distribution is heavy-tailed: a few hub proteins have thousands of neighbors, most have a handful. Expect load imbalance if you give each node one thread.

Common misconception

The interactome is incomplete and biased. Famous proteins (studied in many papers) have more recorded interactions because more people looked, not necessarily because they interact more. This is why the project always compares against degree-matched random sets.

1.3 Alzheimer's disease and the disease module

Alzheimer's disease (AD) causes plaques of amyloid-β (a fragment of the protein APP) and tangles of the protein tau (gene *MAPT*), followed by loss of nerve cells. Large genetic studies compare DNA from tens of thousands of people with and without AD and identify regions where variants change risk; the genes in those regions (e.g., *APOE*, *TREM2*, *BIN1*) are our **AD genes**. On the interactome they are not scattered randomly: they tend to sit in a connected neighborhood, the **disease module**.

1.4 Drugs and targets

Most drugs work by binding one or a few specific proteins, called the drug's **targets**, and blocking or activating them. Databases such as ChEMBL, DGIdb, and DrugBank list known drug–target pairs. For the GPU, a drug is just a short list of node IDs.

1.5 Drug repurposing

Repurposing means testing an already-approved drug for a new disease. Its safety profile is already known, so trials are cheaper and faster. The computational job is to shortlist which of thousands of drugs deserve that test.

2 Network proximity, step by step

2.1 Shortest paths

The distance $d(u, v)$ is the smallest number of edges from u to v. On an unweighted graph, one BFS from u gives $d(u, \cdot)$ for every node. One BFS from every node gives the full $N \times N$ distance matrix D.

Hardware analogy

Interactome distances are tiny (a "small world"): most pairs are 2–5 steps apart and almost none exceed 15, so each entry fits in a `uint8_t`. $18{,}000^2$ bytes is 324 MB: the whole matrix lives on the GPU, and every later question becomes a memory look-up.

2.2 Bit-parallel multi-source BFS

Running 18,000 separate BFS searches wastes work: they all scan the same edges. Multi-source BFS runs 64 searches at once by giving each node a 64-bit word where bit j means "search j has reached this node." Advancing all 64 searches by one level for node v is:

$$\texttt{next}[v] = \Big(\bigvee_{u \in \text{nbrs}(v)} \texttt{frontier}[u] \Big) \ \wedge \ \neg\, \texttt{seen}[v].$$

Use W words per node to run $64W$ searches per batch.

Hardware analogy

This is SIMD inside a register (SWAR): 64 independent searches advance with one OR per edge. On the GPU, one thread per (node, word) gives SIMT parallelism on top of the bit-level parallelism. The inner loop is loads, `OR`, `AND`, `NOT`, and `__ffsll` to find newly set bits.

2.3 Proximity

For a drug with targets T and the AD module S: for each target, find the distance to the nearest AD protein, then average:

$$d_c = \frac{1}{|T|} \sum_{t \in T} \min_{s \in S} D[s][t].$$

Small d_c means the drug acts near AD.

2.4 Why random samples, and what a z-score is

A drug that hits a hub protein is close to everything, so a small d_c alone proves little. We ask: *how small would d_c be by chance for a drug like this?* We build R random "fake drugs" and "fake AD modules" by swapping each real protein for a random protein with a similar number of connections (same *degree bin*), and compute d_c for each. These R values have mean μ and standard deviation σ. Then

$$z = \frac{d_c - \mu}{\sigma}$$

says how many standard deviations the real drug is from chance. $z = -3$ means much closer than chance; $z \approx 0$ means ordinary.

Hardware analogy

Each random sample is independent: draw random node IDs, gather distances from D, take minimums, average. Thousands of drugs times thousands of samples is millions of identical, independent tasks: a textbook GPU workload, limited by scattered memory reads rather than arithmetic. Using a counter-based RNG (Philox) keyed by (drug, sample) makes the CPU and GPU draw identical numbers, so results can be compared exactly.

2.5 Separation between two drugs

To pick drug *combinations*, we also need to know whether two drugs hit the same neighborhood. The separation s_{AB} compares how close the two target sets are to each other versus how clustered each is on its own. Negative means they overlap (likely redundant); positive means they act on different parts of the network. Good pairs are both close to AD but separated from each other ("complementary exposure").

3 How we know if the results mean anything

3.1 Positives and baselines

We cannot test drugs in patients, so we use a proxy: drugs that experts already considered promising enough to enter AD clinical trials. If our ranking puts many of them near the top, the method is capturing something real. We compare against simple baselines (random order, number of targets) because a method is only useful if it beats the obvious.

3.2 AUROC, AUPRC, and enrichment

- **AUROC**: the probability that a random positive drug is ranked above a random non-positive one. 0.5 is chance, 1.0 is perfect.
- **AUPRC**: area under the precision–recall curve; more informative when positives are rare (they are).
- **Enrichment in the top k**: how many positives are in the top 50 or 100 compared with what chance predicts, with a hypergeometric p-value.

Common misconception

Circularity: many AD trial drugs target amyloid or tau directly, and those proteins are in our AD module, so they are trivially "close." That proves nothing. We therefore also report results after removing drugs that target AD-module genes directly.

4 Optimization: QUBO

A **Quadratic Unconstrained Binary Optimization** problem is

$$\min_{x\in\{0,1\}^n} x^\top Q x = \sum_i Q_{ii}x_i + \sum_{i<j} Q_{ij}x_i x_j.$$

Linear terms sit on the diagonal because $x_i^2 = x_i$. Constraints ("choose exactly k") become penalty terms such as $A(\sum_i x_i - k)^2$.

For drug combinations, $x_i = 1$ means "include drug i." The diagonal rewards drugs that are close to AD; off-diagonal terms penalize pairs whose targets overlap.

Hardware analogy

Brute force is a loop over all 2^n values of an n-bit integer. In Gray-code order consecutive values differ by one bit, so the cost updates in $O(n)$ instead of $O(n^2)$. One GPU thread per block of patterns, then a min-reduction. At $n = 28$ that is 2.7×10^8 patterns: seconds on a GPU. This exact answer is the yardstick for the quantum method.

5 Quantum computing, just enough

5.1 Qubits and the state vector

An n-qubit register is described by 2^n complex numbers (amplitudes) $\psi \in \mathbb{C}^{2^n}$ with $\|\psi\|_2 = 1$. Measuring it returns bit pattern b with probability $|\psi_b|^2$.

Hardware analogy

Simulating n qubits means storing ψ explicitly: $2^n \times 8$ bytes at complex64. That is 2 GiB for 28 qubits and 8 GiB for 30; every extra qubit doubles memory. This is why the quantum part only handles a shortlist of about 20–30 drugs (one qubit per drug).

5.2 Gates

A gate is a unitary matrix applied to ψ. A one-qubit gate on qubit q applies the same 2×2 complex matrix to every pair of amplitudes whose indices differ only in bit q. A gate that is diagonal (as in QAOA's cost step) just multiplies each amplitude by a phase.

Hardware analogy

A one-qubit gate is a strided butterfly over the state vector, like one stage of an FFT: pair element i with $i \oplus (1 \ll q)$ and do a 2×2 complex multiply. It is bandwidth-bound and perfectly data-parallel.

5.3 QAOA

The Quantum Approximate Optimization Algorithm approximately solves a QUBO:

1. Put every qubit in an equal superposition (all 2^n bit patterns equally likely).
2. Repeat p times: multiply each amplitude ψ_b by $e^{-i\gamma\,\mathrm{cost}(b)}$ (cost step), then rotate every qubit by an angle β (mixing step).
3. Compute the expected cost $\sum_b |\psi_b|^2\,\mathrm{cost}(b)$.
4. A classical optimizer adjusts the $2p$ angles to lower it; repeat.
5. Measure: the most likely bit patterns are candidate answers.

We judge it by the **approximation ratio** (how close its answer's cost is to the true optimum) and by how often it samples the optimum.

Common misconception

Simulating QAOA on a GPU is *slower* than brute-forcing the same small QUBO. The quantum part is not about speed. It asks how good QAOA's answers are on a real, biology-derived problem, which matters for future quantum hardware.

5.4 cuQuantum and CUDA-Q

cuQuantum is NVIDIA's set of GPU libraries for simulating quantum circuits; it is software, not a quantum computer.

- **cuStateVec** stores ψ in GPU memory and applies gates, computes expectations, and samples measurements. We use this.
- **cuTensorNet** contracts the circuit as a tensor network, which can reach more qubits for shallow circuits.
- **CUDA-Q** is NVIDIA's C++/Python quantum programming framework. You write the circuit there; it calls cuStateVec for you. Qiskit Aer can also use cuQuantum as its GPU backend.

6 Where each concept lives in the pipeline

Module	Biological meaning	Computational meaning	Hardware
1 Data prep	Which proteins interact; which drugs hit which proteins; which genes are linked to AD	Parse tables, map IDs, build CSR, degree bins	CPU
2 Distances	How far apart any two proteins are	Bit-parallel multi-source BFS → `uint8` $N \times N$ matrix	GPU
3 Proximity	Is the drug unusually close to AD?	Random sampling, gathers from D, min/mean, block reductions	GPU
4 Validation	Does the ranking agree with experts' trial choices?	Sort, AUROC/AUPRC, hypergeometric tests	CPU
5 Combinations (optional)	Which few drugs cover AD without redundancy?	Pairwise separation, QUBO, Gray-code brute force, QAOA	GPU + cuQuantum

7 Glossary

AD module
Set of AD-linked proteins on the interactome.

AUROC / AUPRC
Ranking-quality metrics; 0.5 AUROC is chance.

BFS
Breadth-first search; gives shortest paths on unweighted graphs.

BioGRID / STRING
Protein-interaction databases.

ChEMBL / DGIdb / DrugBank
Drug–target databases.

Complementary exposure
Two drugs both close to the disease but separated from each other.

CSR
Compressed sparse row graph format.

cuQuantum / cuStateVec
NVIDIA GPU libraries for simulating quantum circuits.

CUDA-Q
NVIDIA's quantum programming framework.

Degree bin
Group of nodes with similar numbers of neighbors, used for fair random sampling.

GWAS
Genome-wide association study; finds DNA regions tied to disease risk.

Interactome
Graph of all known protein–protein interactions.

MS-BFS
Multi-source BFS with one bit per search.

Philox
Counter-based random number generator; same output on CPU and GPU for the same key.

Proximity (d_c)
Average distance from a drug's targets to the nearest disease protein.

QAOA
Hybrid quantum–classical algorithm for approximate optimization.

QUBO
Minimize $x^\top Qx$ over binary x.

Repurposing
Testing an existing drug for a new disease.

Separation (s_{AB})
Network overlap measure between two protein sets; negative means overlapping.

State vector
The 2^n complex amplitudes describing n qubits.

Target
A protein a drug binds.

z-score
How many standard deviations a value is from the random mean.

Suggested next reading

- Guney et al. (2016), *Network-based in silico drug efficacy screening*, Nature Communications 7:10331—the proximity method.
- Cheng, Kovács & Barabási (2019), *Network-based prediction of drug combinations*, Nature Communications 10:1197.
- Then et al. (2014), *The more the merrier: efficient multi-source graph traversal*, PVLDB 8(4)—bit-parallel MS-BFS.
- Glover, Kochenberger & Du (2019), QUBO tutorial, 4OR 17.
- Farhi, Goldstone & Gutmann (2014), arXiv:1411.4028—QAOA.
- NVIDIA CUDA-Q and cuQuantum documentation (cuStateVec overview).